

From Cloud to Edge: Building Autonomous AI Agents with Gemini, ADK and On-Device AI

Daniele Mario Areddu · Backend & AI Developer

GDG Chişinău — Chişinău, Moldova



Google Developer Group
GDG Chişinău

**Daniele Mario
Areddu**



Engineering AI beyond the demo

Daniele Mario Areddu

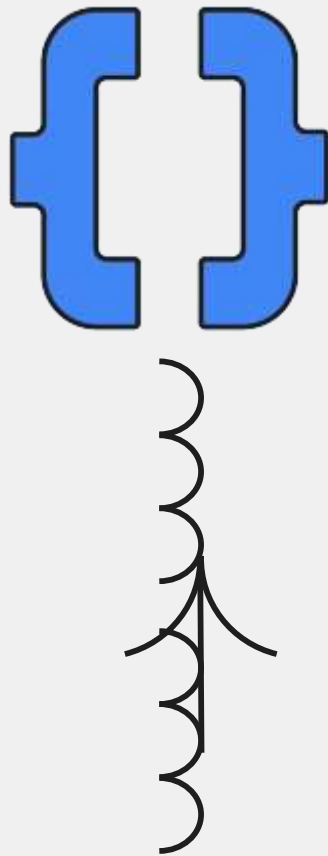
Backend & AI Developer · Global Technologies Italia

Computer Science Student · University of Calabria

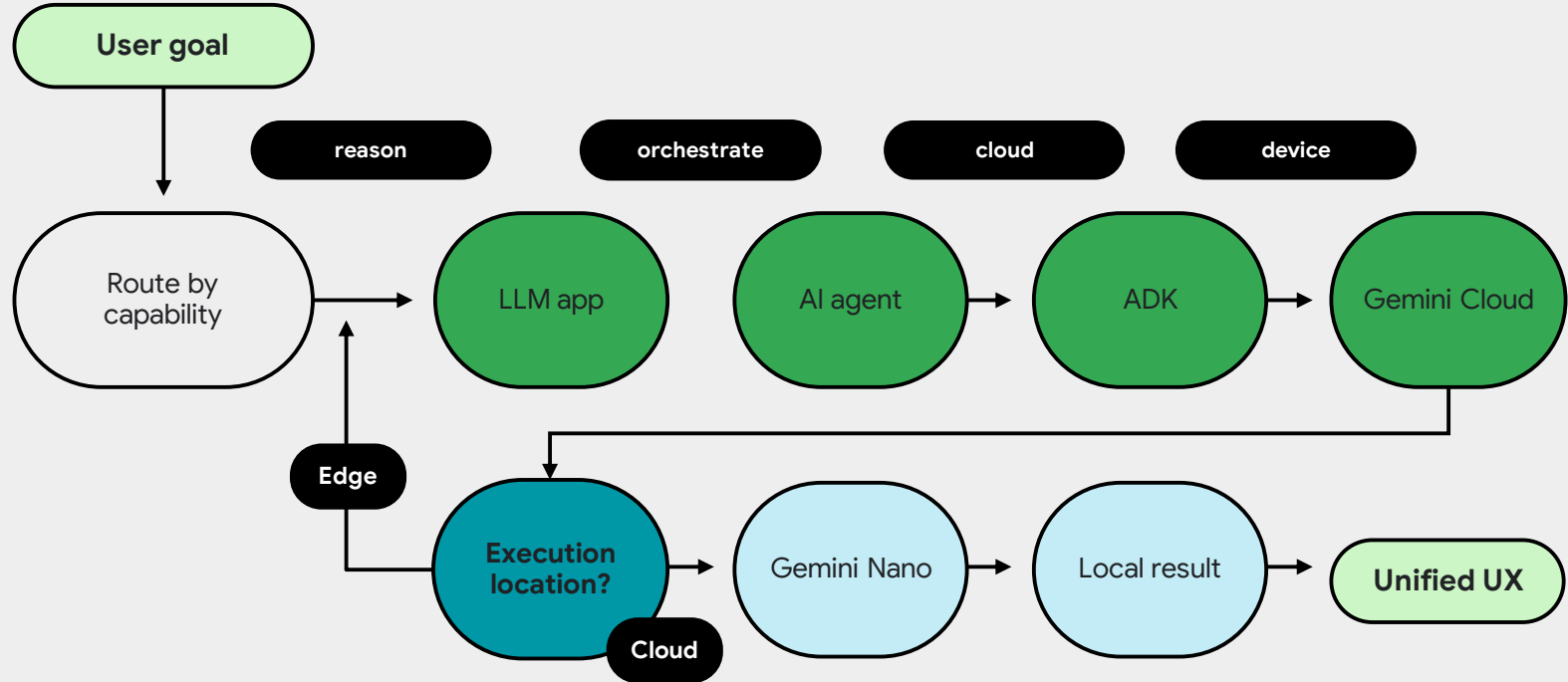
Founder & Community Lead · Velletri.dev

Applied AI · backend systems · data platforms

Models can reason. Architecture stays in control.



Autonomous agents span cloud and device



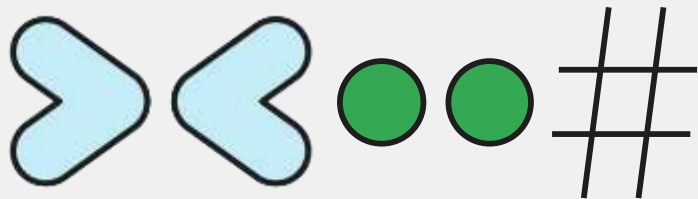
Agents pursue outcomes—not just responses

LLM application

Single prompt · stateless

One response · no planning

Manual tool usage



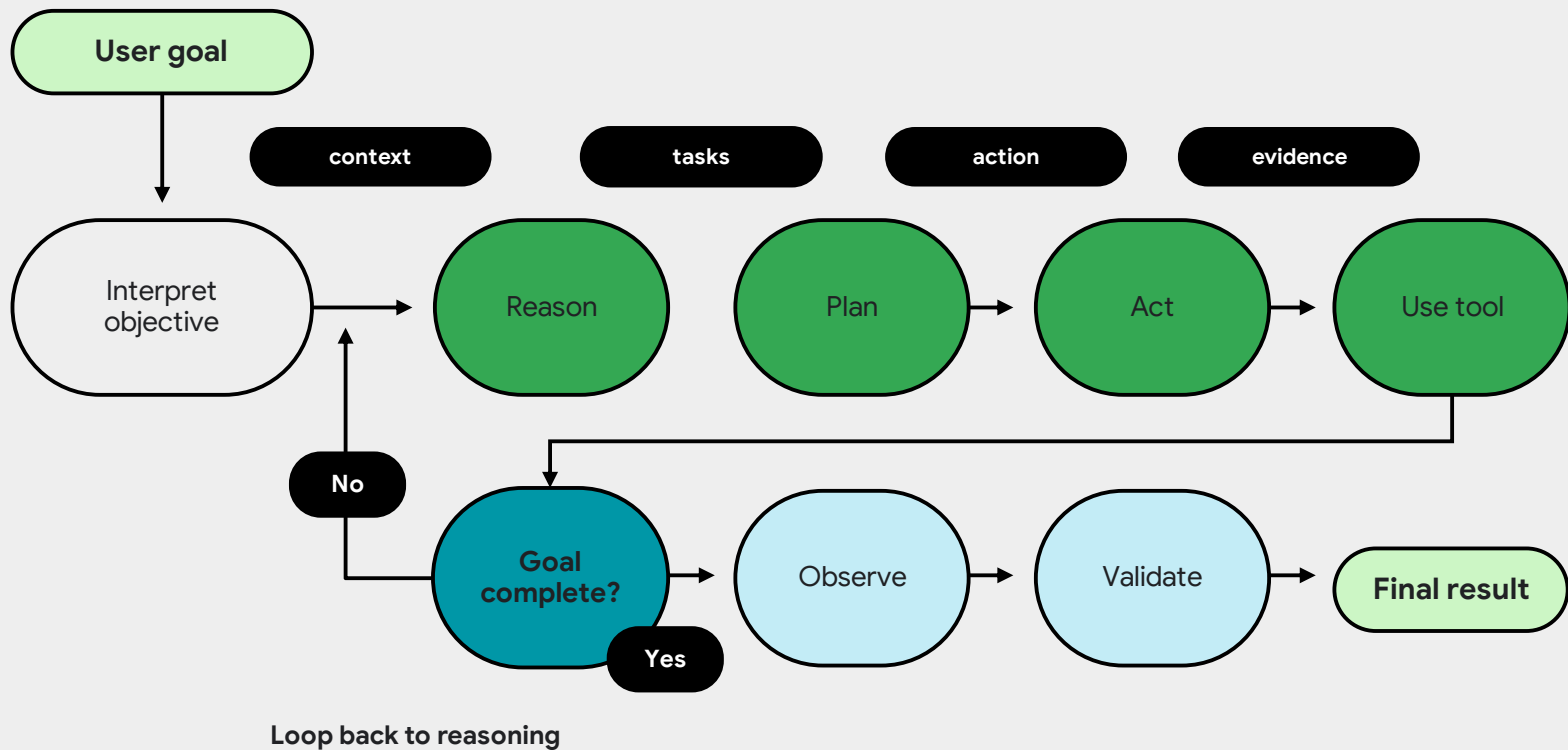
AI agent

Goal-driven · persistent

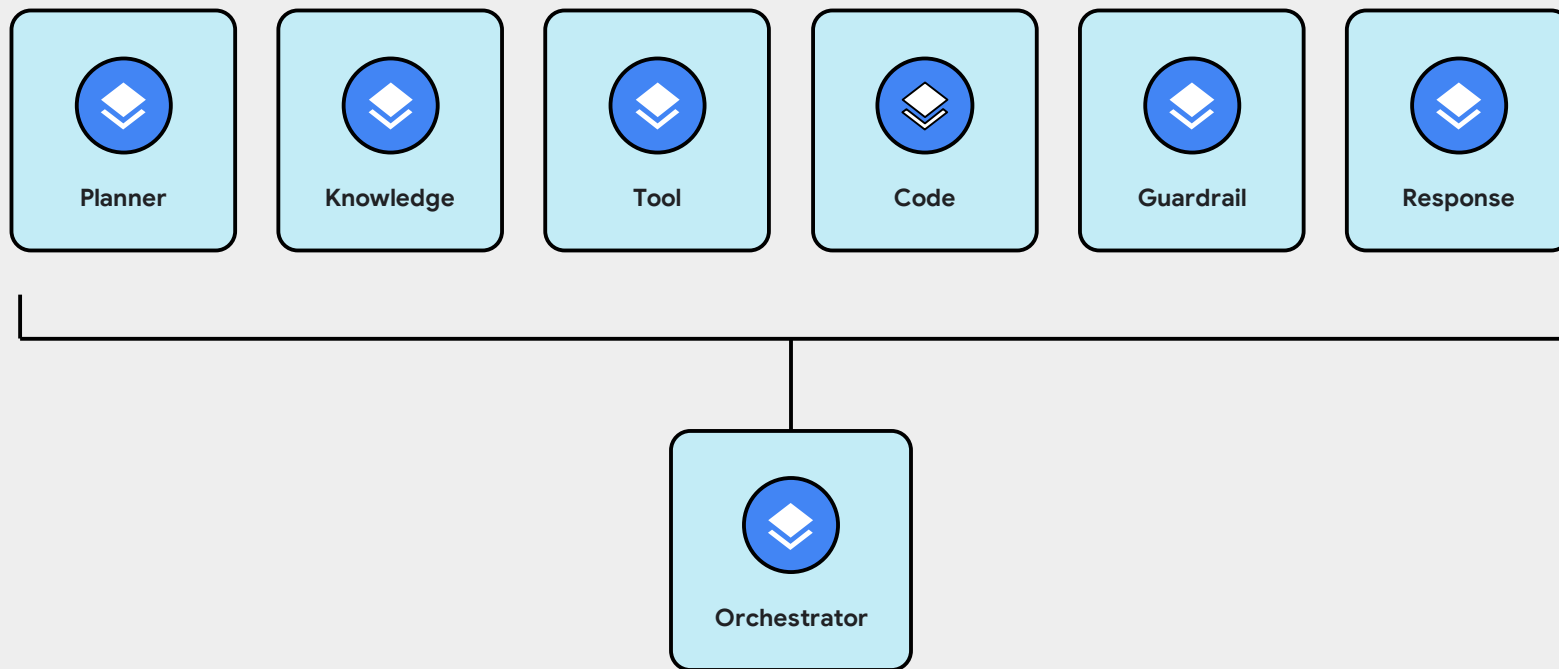
Multi-step · dynamic planning

Autonomous tool calling

Reliable agents repeat reason → act → observe



One orchestrator coordinates specialized agents



Route each task to the right Gemini model



Gemini Nano

Mobile & edge
Privacy-first
Offline capable

Gemini Flash

Low latency
High throughput
Cost-efficient APIs

Gemini Pro

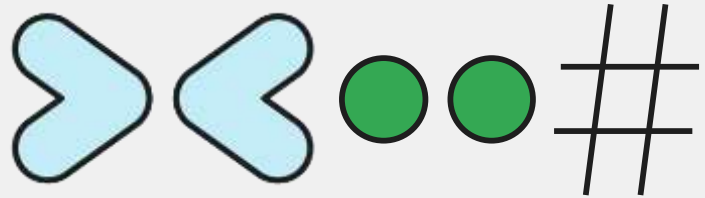
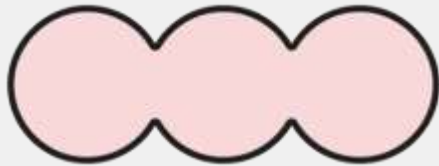
Advanced reasoning
Complex agents
Large workflows

MODEL ROUTING

Flash optimizes flow; Pro handles depth

Gemini Flash

Low latency
Lower cost
High throughput
Good fit for APIs



Gemini Pro

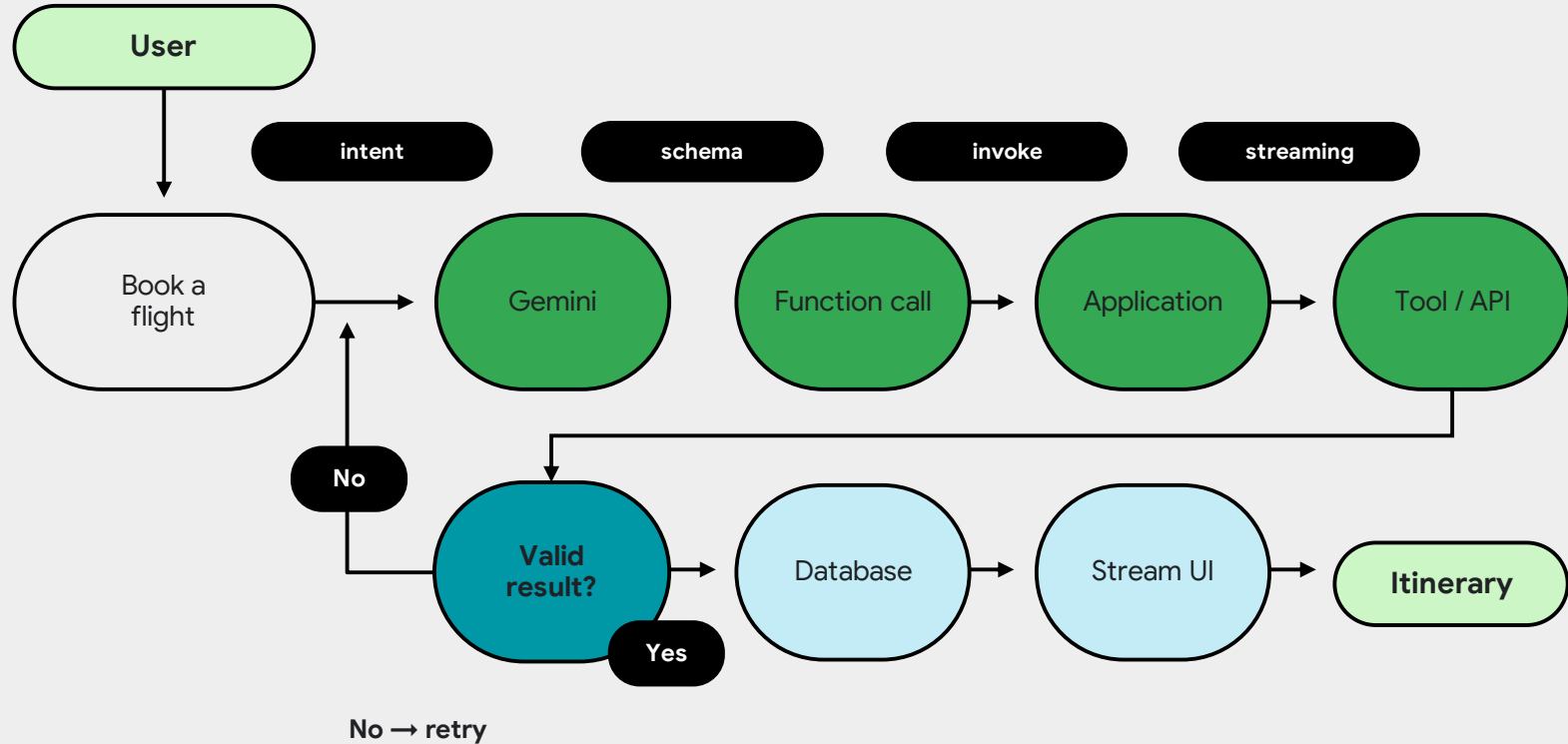
Advanced reasoning
Agent planning
Large workflows
Good fit for complex agents

Structured outputs turn text into reliable systems



```
{  
  "type": "object",  
  "properties": {  
    "priority": { "type": "string" },  
    "assignee": { "type": "string" },  
    "deadline": { "type": "string" }  
  },  
  "required": ["priority", "assignee", "deadline"]  
}
```

Function calls connect reasoning to real systems





ADK separates reasoning, state and capability

Reason

Agent
Planner
Gemini model

State

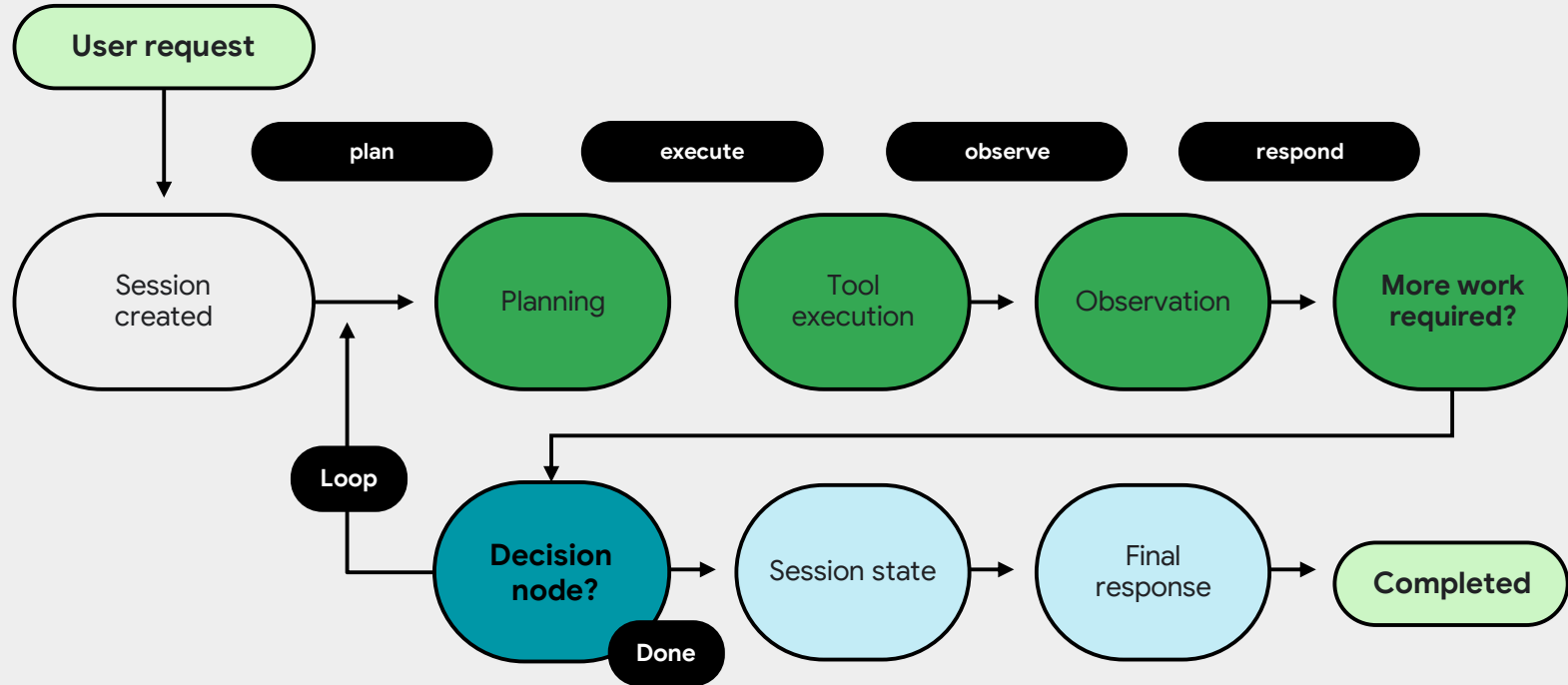
Session
Memory
Context persistence

Act

Tools
External APIs
Structured response

AGENT RUNTIME

An ADK run has a controlled lifecycle



Sessions track execution; memory carries context forward

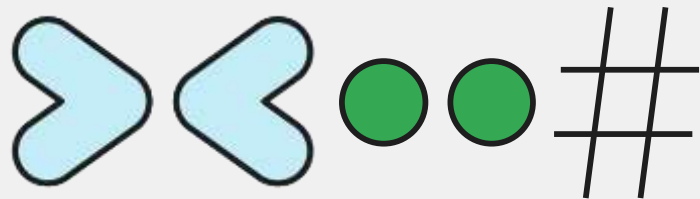
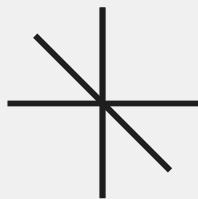
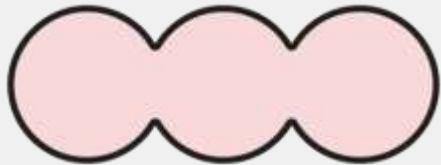
Session

Current execution context

Duration depends on the run

Workflow state

Retention depends on backend



Memory

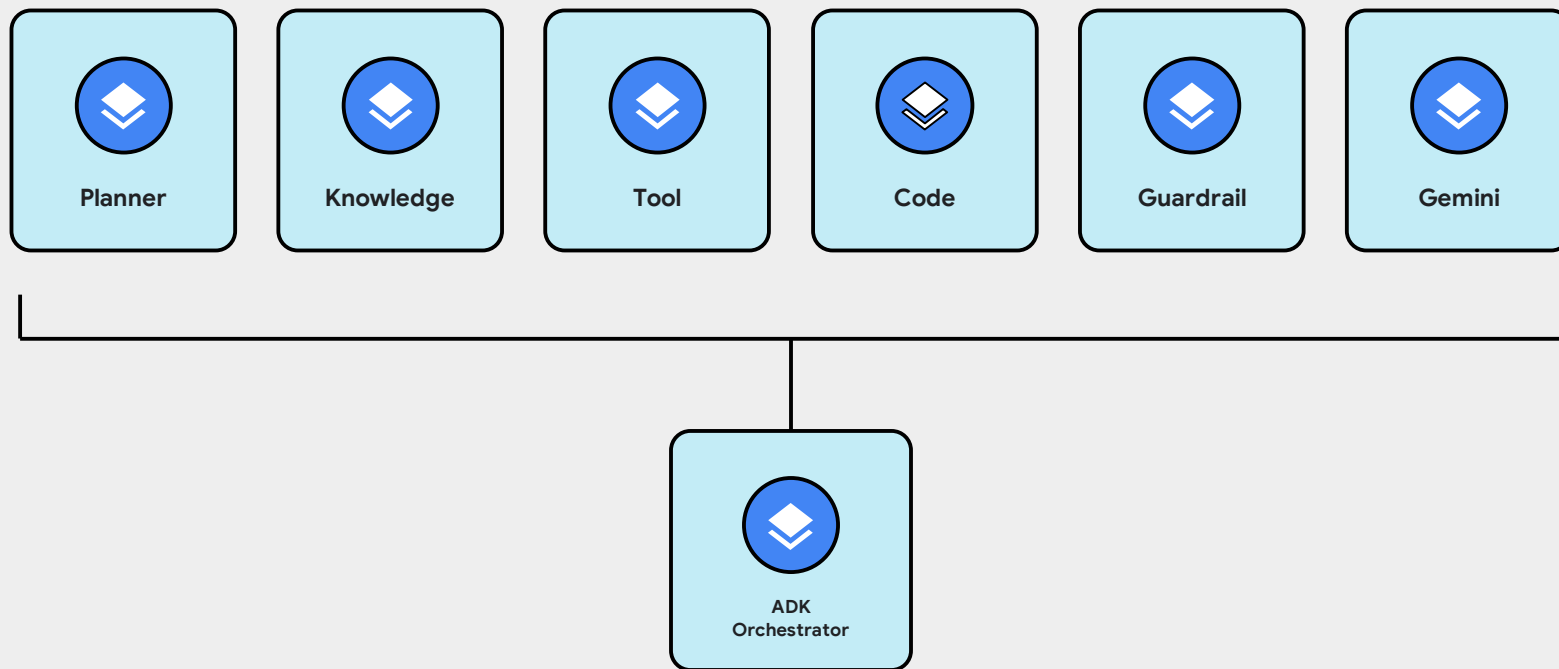
Persistent · across conversations

Lives weeks or months

User knowledge

Retrieved when needed

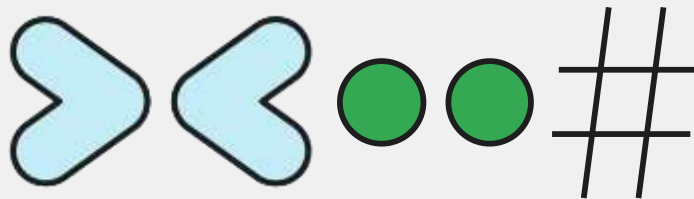
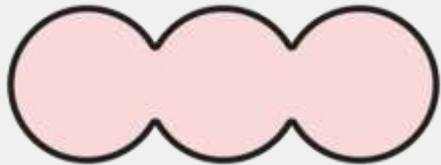
ADK composes agents around clear responsibilities



MCP standardizes many integrations

Traditional integration

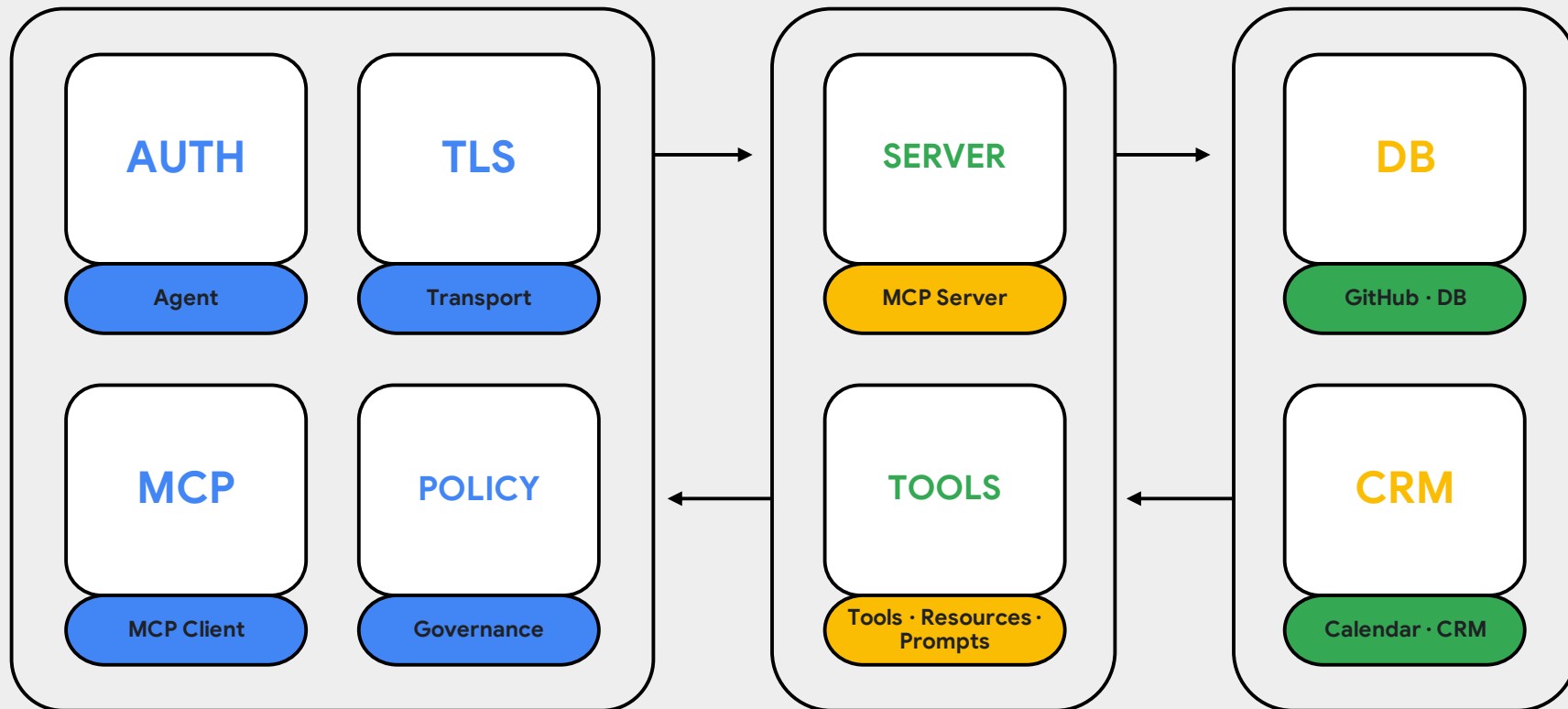
Custom connector per API
Tight coupling
Manual integrations
Hard to maintain



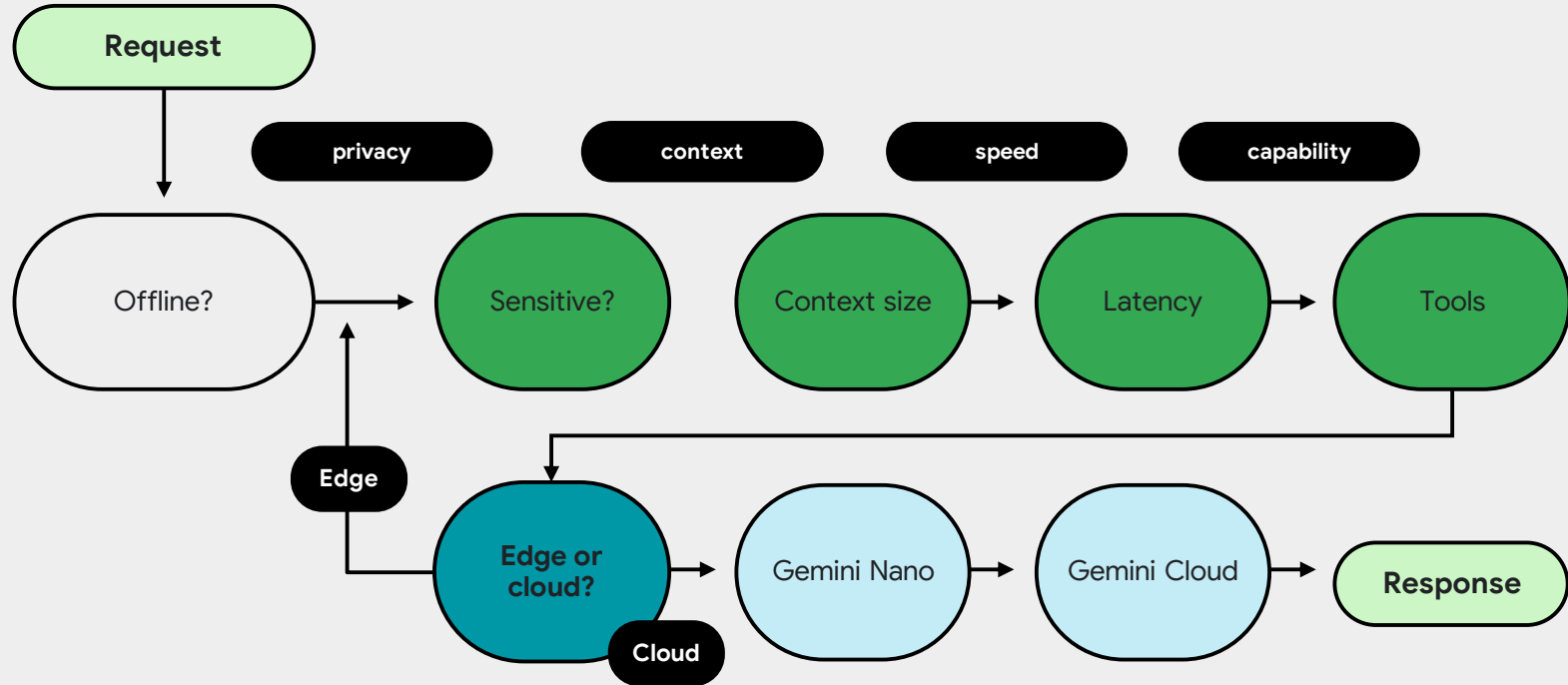
Model Context Protocol

Common interoperability layer
Looser coupling
Tool discovery
Easier to extend

MCP clients, servers and secure tool discovery



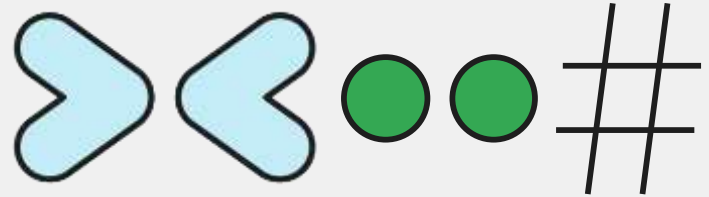
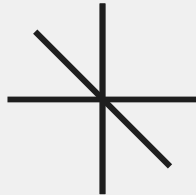
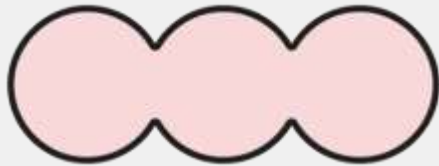
Route by privacy, latency and task complexity



Edge wins immediacy; cloud wins depth

Edge AI

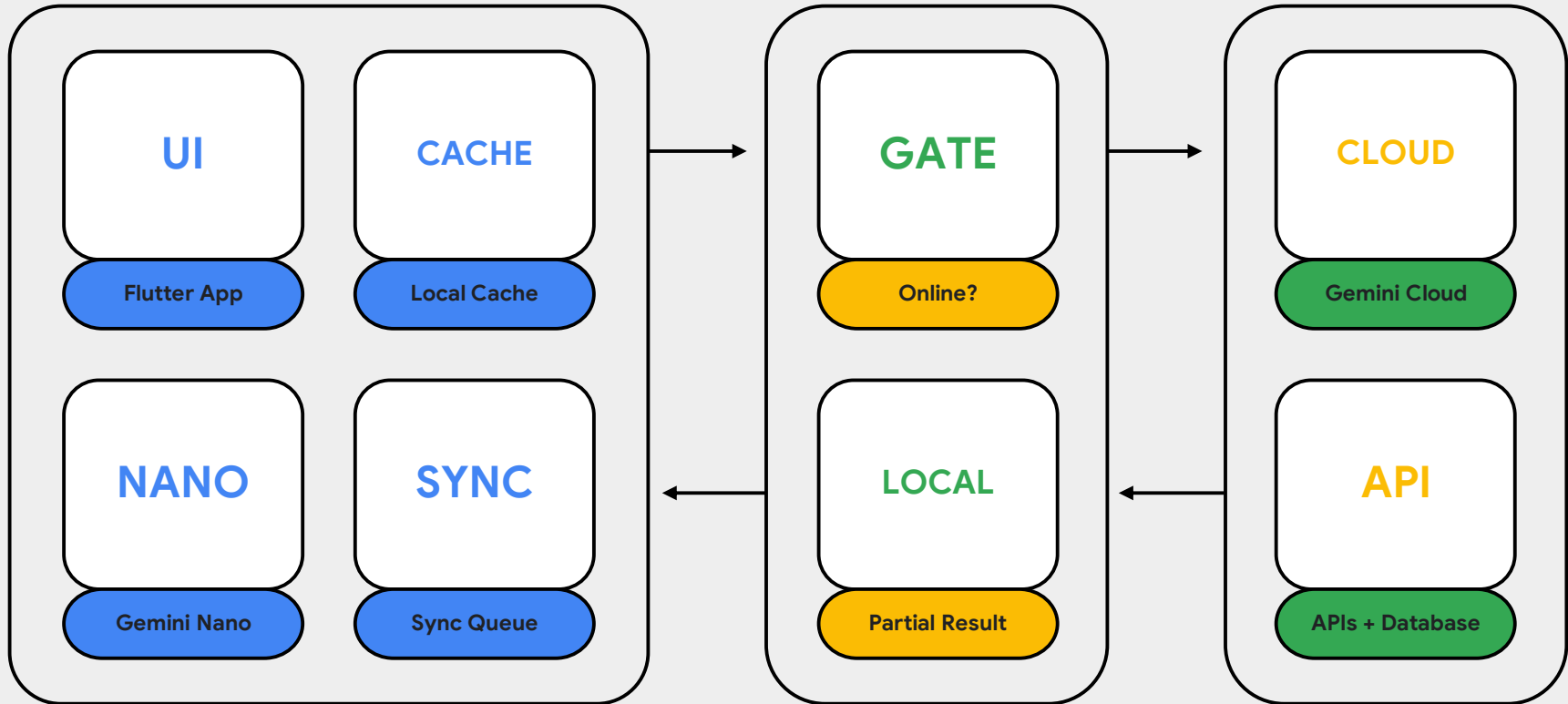
Low latency · offline
Can keep data local
Battery constrained
Device resources



Cloud AI

Advanced reasoning
Elastic compute · quotas apply
Enterprise integrations
Network dependent

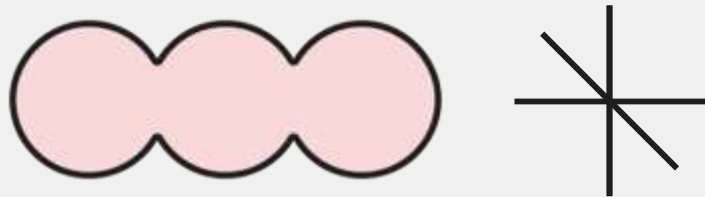
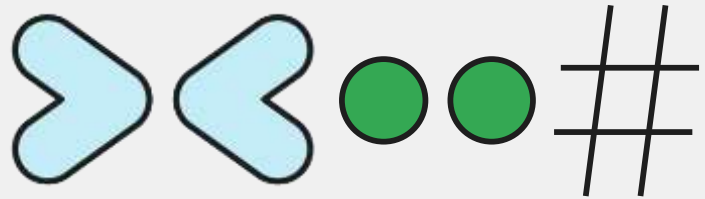
An edge-first app keeps working without a network



Production systems engineer context—not just prompts

Prompt engineering

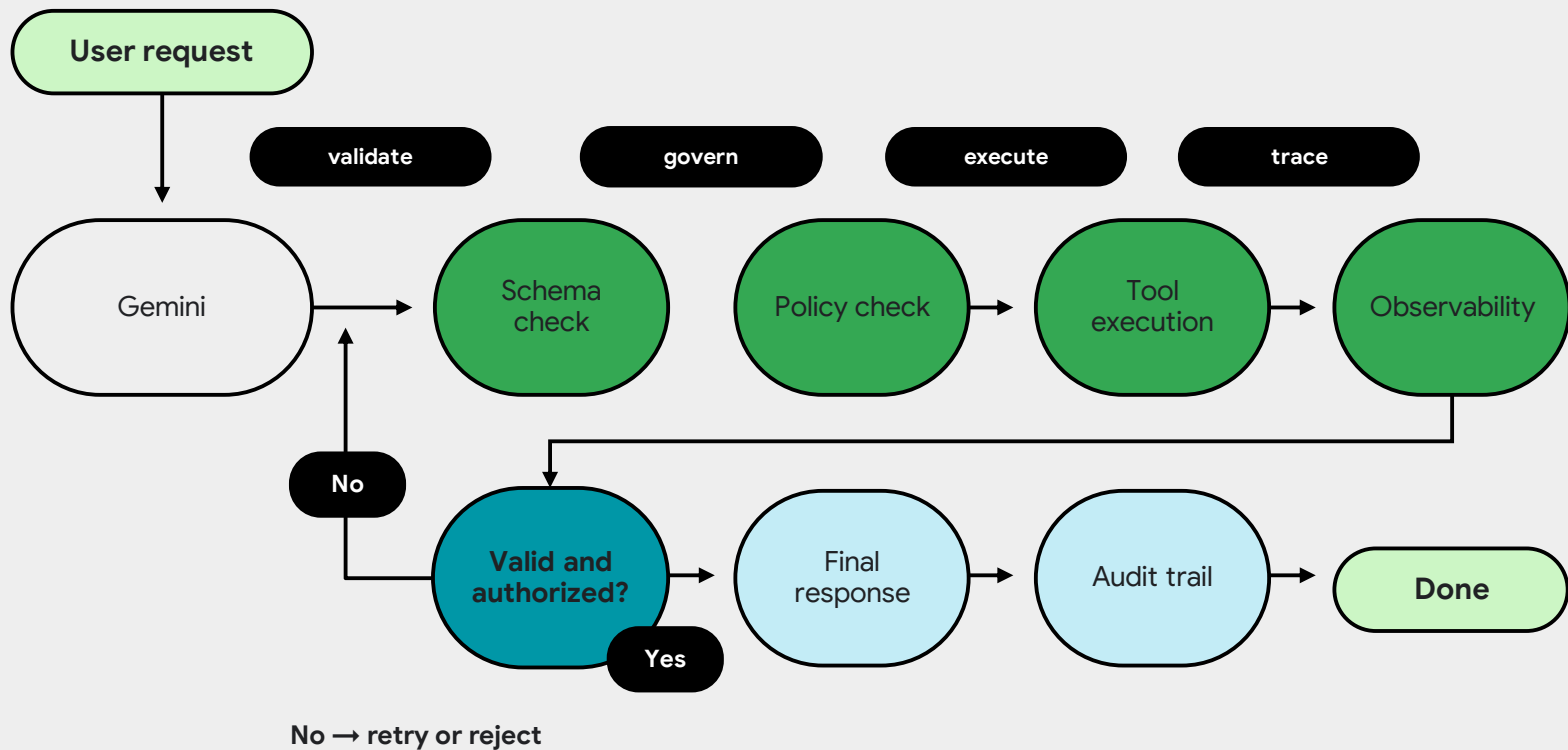
- Better wording
- Static prompt
- Manual optimization
- Single interaction



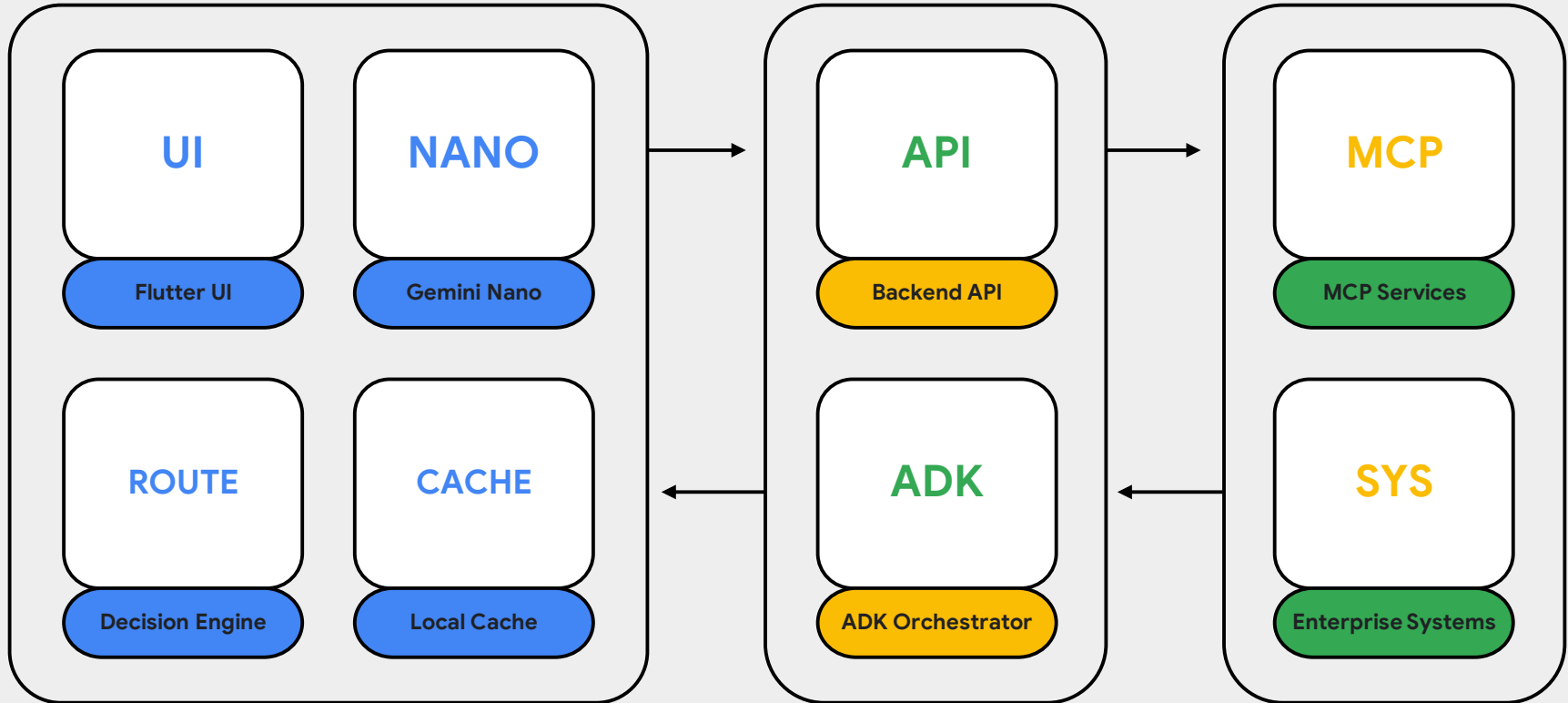
Context engineering

- Better information
- Dynamic context
- Automated retrieval
- Iterative reasoning

Reliability is an explicit control loop



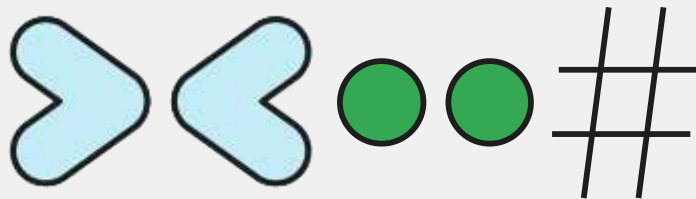
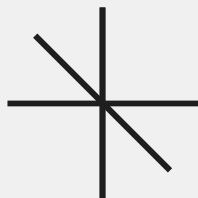
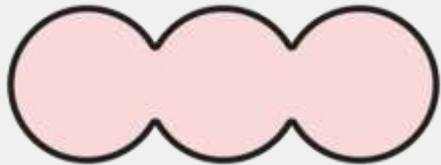
A production agent is a system of systems



Production maturity comes from engineering choices

Anti-pattern

- One giant prompt
- One model for everything
- Free-text outputs
- No monitoring
- Tight API coupling



Best practice

- Agent orchestration
- Intelligent routing
- JSON Schema
- Full observability
- MCP integrations

Thank you Questions?

LinkedIn: Daniele Mario Areddu
Instagram: @danieleareddu_

